

# Segurança e Privacidade

## *ASSIGNMENT #1: Performance Benchmarking of Cryptographic Mechanisms*

Alunos: André Chen Xie, Beatriz Morais Vieira, Manuel Henrique da Silva Mota

Licenciatura: Inteligência Artificial e Ciências de Dados

Data: 30 de março de 2026

## Introdução

O objetivo deste relatório é avaliar e comparar o desempenho computacional de diferentes mecanismos baseados em criptografias simétrica, assimétrica e funções de dispersão (*hash*). Especificamente, são analisados os tempos de execução do AES em modo CTR (chave de 256 bits), de um esquema de encriptação RSA (chave de 2048 bits) e da geração de resumos criptográficos com SHA-256.

A nossa análise baseou-se em ficheiros de texto de 8 bytes a 2 MB. Para garantir a validade estatística dos resultados obtidos, tendo sido aplicada uma abordagem rigorosa na medição dos tempos. O objetivo desta avaliação é justificar as diferenças de desempenho encontradas, dando especial atenção à comparação entre os métodos simétricos e assimétricos, bem como à diferença de tempo gasto a encriptar e a desencriptar dados com o algoritmo RSA.

## Setup Experimental e Metodologia

Foi necessário algum cuidado metodológico aqui, porque nas primeiras execuções os resultados variavam bastante — o que nos levou a investigar o impacto do *garbage collector* e do *Warm-up*.

## Características do Computador:

- Sistema Operatório: Ubuntu 24.04.2 LTS
- Versão de Software: Visual Studio Code version 1.112.0
- Kernel selecionado: Python 3.13.11
- Processador: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz
- Arquitetura de Sistema: x86\_64 (64 bits)
- RAM: 16,00 GB

## Rigor no Benchmarking:

A nossa metodologia focou-se em medir estritamente o tempo de processamento criptográfico, seguindo estes princípios:

1. Isolamento de I/O: Todos os ficheiros foram carregados para a memória RAM (`f.read()`) antes do início da cronometragem, eliminando o tempo de leitura do disco.

2. Warm-up: Executamos o algoritmo algumas vezes (5) antes da medição real para garantir que as instruções estavam na cache do processador.
3. Controlo do Ambiente: foi utilizado o `time.perf_counter_ns()` para máxima resolução e desativamos o Garbage Collector durante os ciclos de teste.
4. Tratamento estatístico: o código calcula o Intervalo de Confiança (95%), permitindo aferir a fiabilidade dos resultados.

## Implementação do código

### Bibliotecas usadas no trabalho

```
import os
import random
import time
import hashlib
import statistics
import math
import gc
import timeit
import matplotlib.pyplot as plt

# --- Importações da biblioteca externa 'cryptography' ---
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import default_backend
```

## EXERCÍCIO A – Geração dos Ficheiros

```
4 def generate_random_text_file(filename, size_bytes):
5
6     #cria um string com todos os caracteres imprimíveis do código ascii (do 32 ao 126)
7     chars = ''.join(chr(i) for i in range(32, 127))
8
9
10    # Gera conteúdo aleatório
11    # Cada caractere ASCII é 1 byte, por isso size_bytes = número de caracteres
12    content = ''.join(random.choice(chars) for _ in range(size_bytes)) #o random.choice escolhe
13    #um caracter pseudoaleatório da string chars
14
15    # Escreve no ficheiro (w = write)
16    with open(filename, 'w', encoding='utf-8') as f: #abre o ficheiro com o nome desejado
17        f.write(content) #escreve no ficheiro
18
19    # Verificar tamanho
20    actual_size = os.path.getsize(filename) #sugestão do chat
21    print(f"Ficheiro '{filename}' gerado: {actual_size} bytes ")
22
```

Este código cria ficheiros e selecciona caracteres imprimíveis da tabela ASCII de forma pseudoaleatória até que o ficheiro tenha o correspondente. Tais tamanhos são: 8B, 64B, 512B, 4096B, 32768, 262144, 2097152.

## EXERCÍCIO B – AES (Advanced Encryption Standard)

```

18 # --- ENCRIPTAÇÃO ---
19 # Criamos o objeto fora para medir APENAS o processamento
20 cifra_enc = Cipher(algorithms.AES(chave), modes.CTR(nonce), backend=backend)
21
22 def tarefa_encryptar():
23     encryptor = cifra_enc.encryptor()
24     return encryptor.update(dados) + encryptor.finalize()
25
26 # WARM-UP (Aquecimento da Encriptação)
27 for _ in range(5):
28     tarefa_encryptar()
29
30 tempos_enc = []
31
32 # DESATIVAR AQUI (antes de começar a medir)
33 gc.disable()
34
35 for _ in range(n_iteracoes):
36     t0 = time.perf_counter_ns()
37     tarefa_encryptar()
38     t1 = time.perf_counter_ns()
39     tempos_enc.append((t1 - t0) / 1000) # Guardar em microsegundos
40
41 # ATIVAR AQUI (mal termine a medição)
42 gc.enable()

```

O código acima ilustra a nossa abordagem para medir os tempos do AES. Para isolar o tempo exato da operação, a cifra (`cifra_enc`) foi criada antes de começar a contagem e foi implementada uma fase de *Warm-up* para estabilizar o desempenho do sistema logo de início. Além disso, o *Garbage Collector*, processo automático que limpa a memória RAM ao eliminar objetos que o código já não usa, podendo assim interferir com as medições, foi desativado durante as mesmas (`gc.disable()`). Por fim, o tempo de execução foi registrado em microssegundos com a função `time.perf_counter_ns()`.

```

44 # --- DECRIPTAÇÃO ---
45 # Preparar o texto cifrado primeiro
46 encryptor_prep = cifra_enc.encryptor()
47 texto_cifrado = encryptor_prep.update(dados) + encryptor_prep.finalize()
48
49 cifra_dec = Cipher(algorithms.AES(chave), modes.CTR(nonce), backend=backend)
50
51 def tarefa_decryptar():
52     decryptor = cifra_dec.decryptor()
53     return decryptor.update(texto_cifrado) + decryptor.finalize()
54
55 # WARM-UP (Aquecimento da Decriptação)
56 for _ in range(5):
57     tarefa_decryptar()
58
59 tempos_dec = []
60
61 # DESATIVAR AQUI
62 gc.disable()
63
64 for _ in range(n_iteracoes):
65     t0 = time.perf_counter_ns()
66     tarefa_decryptar()
67     t1 = time.perf_counter_ns()
68     tempos_dec.append((t1 - t0) / 1000)
69
70 # ATIVAR AQUI
71 gc.enable()

```

De forma análoga, esta parte do código mostra a medição para a descriptação. O cuidado principal aqui foi gerar o texto cifrado antes de iniciar o cronómetro, para garantir que medíamos estritamente o tempo de decifrar os dados, tendo sido mantida a metodologia da encriptação.

Ao executar o código várias vezes, foi possível observar que os tempos de execução nunca são exatamente iguais, apesar de ser usado o mesmo ficheiro. Isto decorre do facto de enquanto o computador está a correr o código, está simultaneamente a executar outras tarefas em segundo plano, como atualizações do sistema, gestão do WIFI, entre outros. Isto acaba por "roubar" ciclos do processador, fazendo com que o tempo oscile.

Além disso, existe a questão da memória cache: na primeira execução, o computador geralmente é mais lento porque precisa de carregar os dados da memória RAM. Nas execuções seguintes, os dados já estão "preparados" na memória interna do processador (cache) e a operação torna-se mais rápida. Embora o Warm-up já combata este problema, para garantir que os resultados são estatisticamente significativos, foram realizadas um dado número de iterações, de acordo com o tamanho de cada ficheiro. Assim, ao calcular a média, o desvio padrão e o intervalo de confiança com base nas respetivas medições é possível filtrar os 'picos' de tempo causados por outras tarefas do computador, obtendo medidas mais corretas.

```

138     for tamanho in tamanhos:
139
140         #numero de iterações:
141         n_iteracoes = 100 if tamanho <= 4096 else 50 if tamanho <= 32768 else 20
142
143         # 1. Mesmo ficheiro
144         mesmo_ficheiro_enc, _ = benchmark_aes_melhorado(f"random_file_{tamanho}bytes.txt", chave, n_iteracoes)
145
146         # 2. Ficheiros diferentes
147         tempos_enc_diff = []
148
149         for i in range(n_iteracoes):
150             nome_temp = f"temp_{i}_{tamanho}bytes.txt"
151
152             with open(nome_temp, 'wb') as f:
153                 f.write(os.urandom(tamanho))
154
155             stats_enc, _ = benchmark_aes_melhorado(nome_temp, chave, n_iteracoes)
156             tempos_enc_diff.append(stats_enc['media'])
157             os.remove(nome_temp)
158
159         media_enc_diff = statistics.mean(tempos_enc_diff)
160         desvio_enc_diff = statistics.stdev(tempos_enc_diff)

```

Relativamente ao uso de diferentes ficheiros com o mesmo tamanho, foi observado que os tempos permaneciam consistentes. Isto acontece porque o AES (Advanced Encryption Standard) é um algoritmo que processa os dados de forma puramente matemática. Como foi utilizado o modo CTR (Counter), o algoritmo gera um fluxo de chaves (key stream) que é combinado com os dados através de uma operação XOR.

## EXERCÍCIO C – RSA (Rivest-Shamir-Adleman)

```

22 # Função hash
23 def H(i, r):
24     return hashlib.sha256(i.to_bytes(8, "big") + r).digest()
25
26 def xor_bytes(a, b):
27     return bytes(x ^ y for x, y in zip(a, b))

```

Neste bloco, são apresentadas as duas ferramentas que criadas para ajudar a construir o criptograma. A primeira, a função H, encarrega-se de gerar a máscara com o SHA-256. O índice i foi convertido para *bytes* (to\_bytes) para que a soma com o r funcionasse sem erros de compatibilidade. Já a função xor\_bytes foi a solução prática encontrada para implementar o operador  $\oplus$  do enunciado. A mesma faz uso do zip para alinhar os *bytes* da mensagem com os da máscara, aplicando a operação XOR a cada par de forma rápida e direta.

```

29 # Enc(m;r) = RSA(r) || (H(0,r) xor m0) || ...
30 def encriptar(mensagem, n, e, k):
31     r = os.urandom(BLOCK_SIZE) # 32 bytes aleatórios
32
33     # RSA(r)
34     r_int = int.from_bytes(r, "big")
35     rsa_r = pow(r_int, e, n).to_bytes(k, "big")
36
37     # SHA-256 + XOR
38     corpo = bytearray()
39     indice = 0
40
41     for i in range(0, len(mensagem), BLOCK_SIZE):
42         bloco = mensagem[i:i + BLOCK_SIZE]
43         mascara = H(indice, r)
44         corpo.extend(xor_bytes(bloco, mascara[:len(bloco)]))
45         indice += 1
46
47     return rsa_r + bytes(corpo)

```

Nesta função foi implementada a fórmula principal do guião. Primeiro, é gerado e cifrado o valor r com o RSA. Depois, a mensagem é partida em blocos, sendo que a cada bloco é aplicado o XOR com a máscara da função H. O pormenor crucial que diferenciou esta abordagem foi o uso de um bytearray. É perceptível que, como os ficheiros chegam aos 2 MB, a concatenação normal de *bytes* deixava o programa excessivamente lento. Com o .extend() do bytearray, esse problema foi solucionado e a execução ficou muito mais rápida.

```

49 def desencriptar(cifra, n, d, k):
50     rsa_r = cifra[:k]
51     corpo = cifra[k:]
52
53     # Recuperar r com RSA^-1
54     rsa_r_int = int.from_bytes(rsa_r, "big")
55     r_int = pow(rsa_r_int, d, n)
56     r = r_int.to_bytes(BLOCK_SIZE, "big")
57
58     mensagem = bytearray()
59     indice = 0
60
61     for i in range(0, len(corpo), BLOCK_SIZE):
62         bloco = corpo[i:i + BLOCK_SIZE]
63         mascara = H(indice, r)
64         mensagem.extend(xor_bytes(bloco, mascara[:len(bloco)]))
65         indice += 1
66
67     return bytes(mensagem)

```

Para a descriptação, foi aplicado o processo inverso. Inicialmente, a cifra é separada para isolar a componente RSA e a chave privada (d) é utilizada para recuperar o r. Na posse desta semente, o ciclo é replicado: as máscaras foram novamente geradas e o XOR aplicado para extrair a mensagem em claro. A otimização com o bytearray foi preservada para assegurar a mesma eficiência na execução.

É ainda importante mencionar que as mesmas táticas de benchmarking, como o Warm-up e a desativação temporária do garbage collector foram mais uma vez aplicadas no código deste exercício. Quanto ao número de iterações, foram novamente adaptadas ao tamanho dos ficheiros, tendo em mente que o RSA envolve cálculos matemáticos muito grandes, sendo assim muito mais lento que o AES ou o SHA-256.

## EXERCÍCIO D – SHA-256 (Secure Hash Algorithm)

```

10  # 1. LER O FICHEIRO PARA A MEMÓRIA ANTES DO CRONÓMETRO!
11  with open(ficheiro, 'rb') as f:
12      dados = f.read()
13
14  # WARM-UP (Aquecimento do SHA-256)
15  for _ in range(5):
16      h_temp = hashlib.sha256()
17      h_temp.update(dados)
18      h_temp.digest()
19
20  # DESATIVAR GC ANTES DO LOOP
21  gc.disable()
22
23  for _ in range(num_execucoes):
24      # Ligar o cronómetro
25      inicio = time.perf_counter_ns()
26
27      sha256 = hashlib.sha256()
28      sha256.update(dados) # Fazer o hash dos dados que já estão na RAM
29      sha256.digest()      # Finalizar o hash
30
31      # Desligar o cronómetro
32      fim = time.perf_counter_ns()
33
34      tempos.append((fim - inicio) / 1000) # converter para µs
35
36  # ATIVAR GC DEPOIS DO LOOP
37  gc.enable()
38

```

Este módulo foca-se na medição da performance do algoritmo de hashing SHA-256 utilizando a biblioteca nativa hashlib do Python, uma vez que se verificou ser mais rápida que aplicar o comando SSL diretamente. Isto deve-se ao facto de o openssl ter muito overhead, tendo que criar um novo processo, alocar memória para o tal e estabelecer ligação, entre outros, antes de executar o comando em si, ao contrário da função da hashlib, biblioteca em C que corre dentro do mesmo processo.

Como benchmarking, tornaram-se a aplicar as estratégias anteriores.

## Apresentação e Análise de Resultados

Os dados recolhidos foram processados para gerar visualizações gráficas que permitem analisar tendências e comportamentos assintóticos.

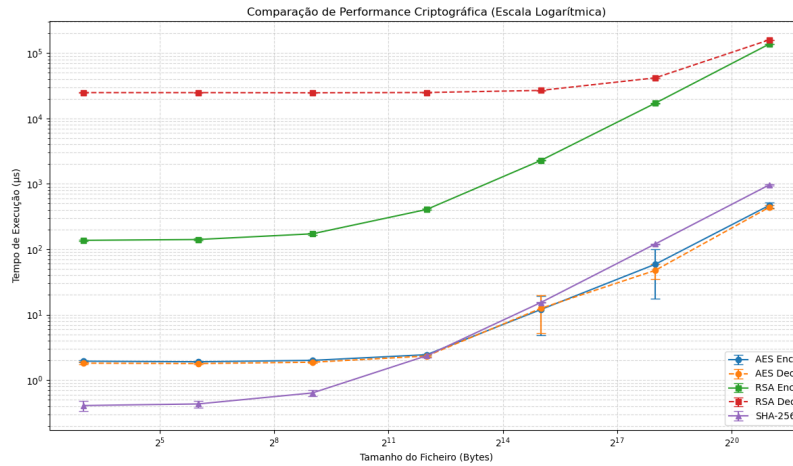


Figura 1: Comparação do tempo de execução dos algoritmos criptográficos em escala logarítmica (base 2 no eixo X e base 10 no eixo Y).

Nota: Os valores exatos de tempo, desvios padrões e os respetivos intervalos de confiança a 95% encontram-se detalhados nas tabelas dos Anexos.

## Visão Geral (Gráfico Log-Log)

A Figura 1 apresenta a comparação macroscópica de todos os algoritmos. O uso de escalas logarítmicas em ambos os eixos é necessário, dada a grande variação entre os ficheiros. Observa-se que todos os algoritmos apresentam um crescimento linear ( $O(N)$ ) em relação ao tamanho do ficheiro (visto como uma linha reta no gráfico log-log). O RSA é consistentemente ordens de grandeza mais lento que o AES e o SHA-256.

## Detalhe em Ficheiros Pequenos: AES vs. SHA-256

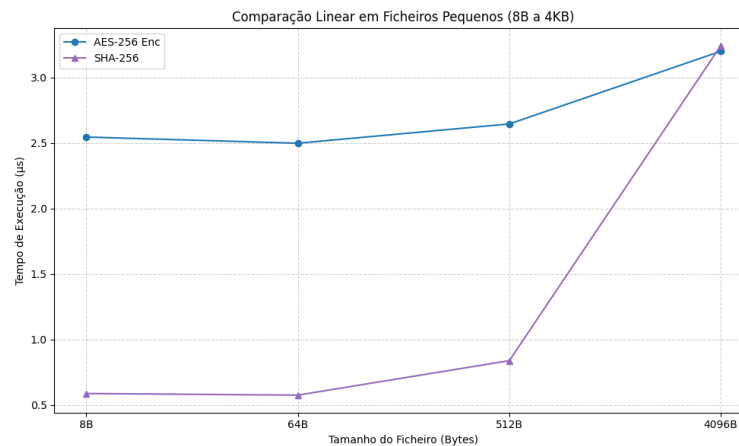


Figura 2: Comparação linear AES vs. SHA-256 em ficheiros pequenos (8B a 4KB).

A Figura 2 foca-se no desempenho em ficheiros de pequena dimensão (até 4 KB). Observa-se que, nas dimensões mais reduzidas (8B, 64B e 512B), o SHA-256 é mais rápido que o AES. Por exemplo, num ficheiro de 8 bytes, o SHA-256 demora apenas ~0.6 µs, contra os ~2.5 µs do AES.

Devido à necessidade de preparar o objeto *Cipher*, expandir a chave e gerar o *nonce*, o AES apresenta um custo fixo inicial que domina o tempo total de execução em ficheiros de pequena dimensão (8 bytes). O SHA-256, por ter uma inicialização menos complexa, demonstra uma performance superior no processamento de volumes reduzidos de dados.

No entanto, à medida que o tamanho do ficheiro aumenta, o tempo de execução do SHA-256 cresce de forma muito mais acentuada do que o do AES. O gráfico mostra que o ponto de convergência ocorre perto dos 4096 bytes (4 KB), onde ambas as operações se cruzam na marca dos  $\sim 3.2 \mu\text{s}$ . Este é o ponto a partir do qual a elevada eficiência de processamento e a aceleração por hardware do AES compensam o seu custo inicial, tornando-o mais rápido que o SHA-256 para ficheiros de maiores dimensões.

## Análise de Throughput

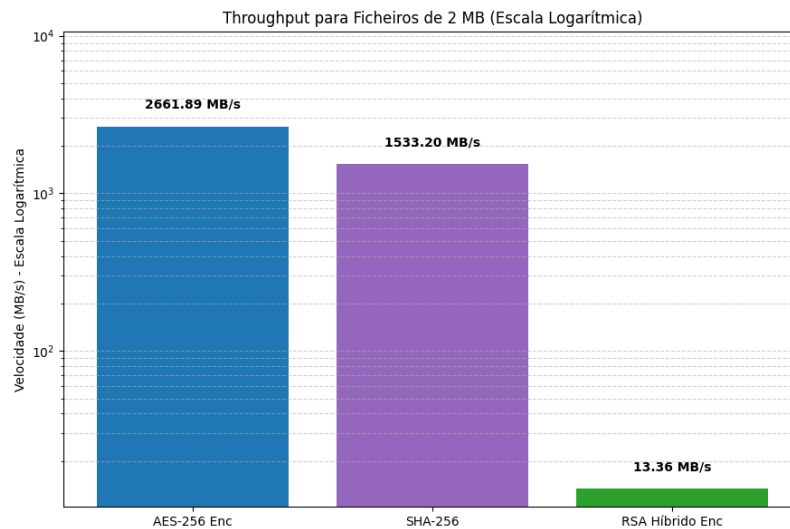


Figura 3: Throughput computacional para ficheiros de 2 MB.

A Figura 3 foca-se na análise da velocidade de processamento (*throughput*) para o ficheiro de maior dimensão testado (2 MB). O gráfico, desenhado com um eixo Y em escala logarítmica para permitir a visualização conjunta dos três valores, demonstra uma diferença de desempenho entre os diferentes mecanismos.

O AES-256 assume a liderança, atingindo uma taxa de processamento de  $\sim 2661.89 \text{ MB/s}$ . Resumidamente, o processador está a usar instruções AES-NI dedicadas, o que faz com que seja tratada como uma operação nativa.

Em segundo lugar surge o SHA-256, alcançando  $\sim 1533.20 \text{ MB/s}$ . Embora seja uma função estruturalmente muito eficiente, depende do poder de cálculo geral do processador, o que lhe impõe um teto de velocidade inferior ao do AES em streams contínuos de grande escala.

Em contraste, a cifra baseada em RSA — mesmo utilizando a variante híbrida otimizada — apresenta um throughput quase residual de apenas  $13.36 \text{ MB/s}$ . Embora a cifragem do corpo da mensagem beneficie da rapidez da operação XOR e da máscara gerada pelo SHA-256, o custo computacional associado à exponenciação modular da chave pública RSA no instante inicial arrasta a velocidade global para baixo.

Isto explica o porquê do TLS não cifrar tudo com RSA. Seria completamente impraticável. O RSA existe para trocar a chave. Depois disso, passa o trabalho ao AES.

## Discrepância de Tempo no RSA (Cifra vs. Decifra)

A Figura 4 ilustra uma assimetria drástica nos tempos de execução dentro do próprio algoritmo RSA. Observando os dados para o ficheiro de menor dimensão (8 bytes), verifica-se que a operação de decifra consome aproximadamente 34 ms, enquanto a cifra do mesmo bloco requer apenas  $\sim 188 \mu\text{s}$ . Isto traduz-se numa operação de decifra quase 180 vezes mais lenta do que a cifra inicial.



A discrepância de performance no RSA reside na natureza dos seus expoentes: a cifragem utiliza o expoente público  $e = 65537$  (relativamente pequeno por design), enquanto a decifragem requer o uso do expoente privado. Como o expoente possui uma extensão próxima dos 2048 bits, exige milhares de multiplicações modulares adicionais, tornando o processo de decifragem computacionalmente muito mais dispendioso e lento.

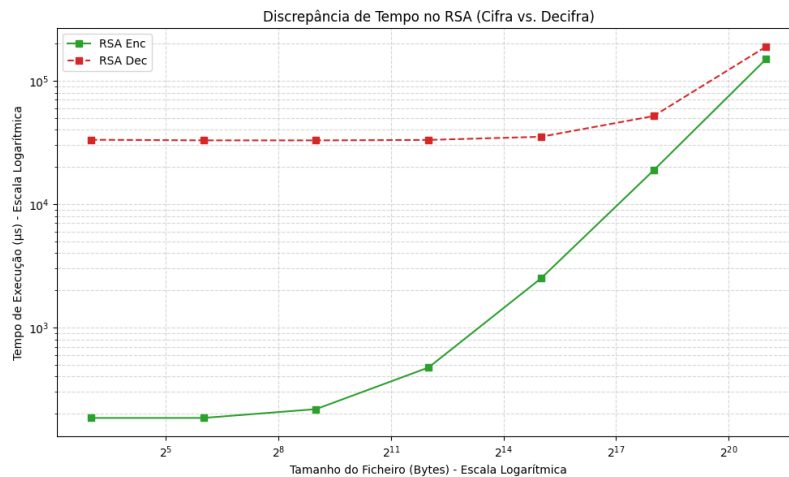


Figura 4: Comparação do tempo de Cifra vs. Decifra RSA.

## Análise da Variabilidade

Para concluir a análise, avaliou-se se o conteúdo do ficheiro (o seu nível de entropia) influencia o desempenho criptográfico. A metodologia consistiu na comparação dos tempos de execução entre a cifragem reiterada de um ficheiro fixo e a cifragem de múltiplos ficheiros com conteúdos aleatórios distintos, mantendo-se constante a dimensão dos dados.

Contrariamente à hipótese de que padrões repetidos poderiam acelerar o processo, os resultados demonstraram ser praticamente idênticos. Por exemplo, no algoritmo AES com blocos de 8 bytes, registaram-se valores de 2.42  $\mu$ s para um ficheiro estático face a 2.40  $\mu$ s para ficheiros variáveis. Esta uniformidade decorre do facto de os algoritmos criptográficos operarem estritamente ao nível binário. No caso específico do AES em modo CTR, o esforço computacional foca-se na geração de um fluxo pseudoaleatório (*keystream*), que é posteriormente combinado com a mensagem através de uma operação lógica XOR. Consequentemente, a carga imposta ao processador permanece rigorosamente a mesma, independentemente de os bytes representarem texto legível, sequências de zeros ou ruído de alta entropia.

O tempo de execução revela-se, assim, exclusivamente uma função do tamanho da entrada, apresentando uma complexidade de  $O(n)$ . As ligeiras variações observadas entre iterações do mesmo ficheiro são atribuíveis a fenómenos de *cache misses* e ao escalonamento de tarefas (*multitasking*) do sistema operativo. A implementação da fase de *warm-up* visou mitigar estes efeitos, sendo o ruído remanescente filtrado através do tratamento estatístico das médias e desvios padrão, calculados sobre uma amostra de um dado número de iterações por teste.

## Conclusão

Os resultados obtidos corroboram as previsões teóricas, destacando-se a grande diferença entre as operações de cifragem e decifragem no RSA, com uma discrepância de performance na ordem das 180 vezes. O AES-256 em modo CTR demonstrou ser uma boa solução para a proteção de dados massivos. O SHA-256 revelou-se uma ferramenta de hashing rápida, superando o AES em mensagens curtas. O RSA confirmou ser computacionalmente pesado, não justificando a sua utilização para cifrar o corpo dos dados diretamente.

Em suma, o esquema híbrido faz sentido precisamente porque nenhum algoritmo é bom em tudo: o RSA gere as chaves, o AES lida com o volume de dados.

# Uso de Inteligência Artificial

A inteligência artificial foi nomeadamente usada para compreender como aplicar as funções das bibliotecas importadas, ou seja, os seus parâmetros e retornos, além dos websites de documentação. Além disso, foi também usada para algumas sugestões de benchmarking, sendo estas a adaptação do número de iterações ao tamanho de ficheiro, ao invés de um número fixo, assim como a desativação do Garbage Collector.

Por fim, auxiliou na geração dos gráficos e na correção de erros no código.

## Anexos: Tabelas de Resultados Usados para Análise

Tabela 1: Desempenho do AES-256 em Modo CTR (Tempos em  $\mu$ s):

| Tamanho (Bytes) | Enc. Média | Enc. Dev. Padrão | Enc. IC 95%      | Dec. Média | Dec. Dev. Padrão | Dec. IC 95%      |
|-----------------|------------|------------------|------------------|------------|------------------|------------------|
| 8               | 2.46       | 0.20             | [2.39, 2.53]     | 2.31       | 0.09             | [2.28, 2.34]     |
| 64              | 3.52       | 2.48             | [2.63, 4.40]     | 2.81       | 0.06             | [2.79, 2.83]     |
| 512             | 3.23       | 0.06             | [3.21, 3.25]     | 3.04       | 0.03             | [3.03, 3.05]     |
| 4096            | 5.11       | 6.81             | [2.67, 7.54]     | 4.97       | 3.97             | [3.54, 6.39]     |
| 32768           | 18.84      | 23.03            | [10.60, 27.08]   | 15.40      | 8.77             | [12.26, 18.54]   |
| 262144          | 89.62      | 57.96            | [68.88, 110.36]  | 53.78      | 12.85            | [49.18, 58.38]   |
| 2097152         | 669.95     | 54.61            | [650.41, 689.49] | 617.08     | 36.74            | [603.93, 630.22] |

Tabela 2: Desempenho do RSA-2048 Híbrido (Tempos em  $\mu$ s):

| Tamanho (Bytes) | Enc. Média | Enc. Dev. Padrão | IC 95% - Enc.          | Dec. Média | Dec. Dev. Padrão | IC 95% - Dec.          |
|-----------------|------------|------------------|------------------------|------------|------------------|------------------------|
| 8               | 135.06     | 3.78             | [133.98, 136.13]       | 24593.17   | 81.88            | [23961.53, 24008.06]   |
| 64              | 137.99     | 3.82             | [136.91, 139.08]       | 24086.83   | 204.25           | [24028.79, 24144.86]   |
| 512             | 168.21     | 4.74             | [166.86, 169.56]       | 24246.80   | 296.31           | [24162.61, 24330.98]   |
| 4096            | 409.20     | 12.93            | [403.15, 415.25]       | 24403.81   | 97.90            | [24358.00, 24449.63]   |
| 32768           | 2310.29    | 61.27            | [2266.47, 2354.12]     | 26306.92   | 137.11           | [26208.84, 26404.99]   |
| 262144          | 17279.65   | 71.14            | [17228.76, 17330.53]   | 41361.52   | 238.62           | [41190.83, 41532.20]   |
| 2097152         | 136081.38  | 426.59           | [135776.23, 136386.52] | 161648.25  | 733.61           | [161123.49, 162173.01] |

Tabela 3: Desempenho do Hash SHA-256 (Tempos em  $\mu$ s):

| Tamanho (Bytes) | Média   | Desvio Padrão | IC 95%             |
|-----------------|---------|---------------|--------------------|
| 8               | 0.58    | 0.09          | [0.56, 0.60]       |
| 64              | 0.56    | 0.03          | [0.55, 0.57]       |
| 512             | 0.83    | 0.03          | [0.83, 0.84]       |
| 4096            | 3.18    | 0.50          | [3.08, 3.28]       |
| 32768           | 24.41   | 8.45          | [22.07, 26.75]     |
| 262144          | 167.50  | 13.28         | [161.68, 173.32]   |
| 2097152         | 1361.25 | 86.28         | [1323.44, 1399.06] |

Tabela 4: Análise de Throughput e Crescimento:

| Tamanho (Bytes) | AES Enc (MB/s) | RSA Enc (MB/s) | SHA-256 (MB/s) |
|-----------------|----------------|----------------|----------------|
| 8               | 3.10           | 0.04           | 13.15          |
| 64              | 17.36          | 0.32           | 109.16         |
| 512             | 151.02         | 2.15           | 587.40         |
| 4096            | 764.68         | 8.00           | 1229.36        |
| 32768           | 1658.56        | 12.25          | 1280.30        |
| 262144          | 2789.44        | 13.51          | 1492.52        |
| 2097152         | 2985.30        | 12.97          | 1469.24        |